

Programación – modelo A

EXAMEN PRÁCTICO TEMA 2 – ESTRUCTURAS DE CONTROL

(31/10/2025)



LEE ATENTAMENTE LAS SIGUIENTES INSTRUCCIONES ANTES DE EMPEZAR:



- **Recopila en un documento de texto las evidencias de todo el examen.** Guárdalo de vez en cuando para no perder el avance de tu trabajo.
- Cuando termines, **pásalo a PDF y sube el documento creado a la entrega de AULES.**

PARTE 1: Configuración del entorno (0,5p)

1. Crea un nuevo repositorio llamado “EXAMEN_UD2_[tu nombre]” desde *SourceTree*. El repositorio debe crearse en local y tener su espejo en remoto, por lo tanto, sincronízalo con *GitHub*.

Haz una captura de pantalla desde la carpeta Local en tu equipo, *SourceTree* y *GitHub* donde se vea el repositorio creado. Pégalas a continuación:

Pega a continuación la URL a tu nuevo repositorio de *GitHub*:

2. Crea un nuevo proyecto *Java* (*Maven*) con *IntelliJ* -o el IDE que utilices- dentro del repositorio que acabas de crear. Llámalo “EXAMEN UD2”.
3. Crea en el proyecto dos nuevas clases *Java*, una para cada ejercicio. Las puedes llamar “Fakejack” y “Vocales”.
4. Añade al método *main()* a las dos clases creadas para poder ejecutar los ejercicios que vas a programar a continuación.

Haz dos capturas de pantalla donde se vean las clases nuevas y sus métodos *main()*. Pégalas aquí:

PARTE 2: Resolución de problemas

Programa en *Java* la solución a los siguientes ejercicios. Usa el proyecto que te acabas de crear en el apartado anterior. **Si no has conseguido crearlo correctamente, utiliza alguno de los proyectos que ya tenías para los ejercicios de clase y pega la URL de *GitHub* del repositorio al que vas a subir los cambios.**

1. (5p) Programa Juego *Fakejack*

El alumnado de 2º de Bachillerato del IES MUTXAMEL necesita obtener dinero para poder irse de viaje de fin de curso a Calasparra del Sur. Como tienen claro aquello de que “la banca siempre gana”, se les había ocurrido montar un casino clandestino en los recreos con el juego del *BlackJack*, pero como jefatura de estudios no se lo permite por razones obvias, se han aliado con los estudiantes de 1º DAW para que les desarrollen una *app* que se comporte igual que el juego y así puedan llevar a cabo su plan de forma digital sin que nadie se entere.

Paso 1. La *app* del *FakeJack* debe iniciar dando la bienvenida y generando un número aleatorio entre 17 y 21, que será la puntuación de la banca durante la partida. **No se debe mostrar todavía.**

Paso 2. A continuación, el programa simulará el lanzamiento de una carta al jugador. Deberá ser un número aleatorio generado entre 2 y 11. Una vez generada la carta, mostrará su valor por pantalla junto a la puntuación actual.

```
*** BIENVENIDO AL FACKEJACK ***  
Tu carta es: 7  
Puntuación actual: 7
```

Paso 3. El programa preguntará al usuario si quiere lanzar otra carta.

➔ Por si no sabes jugar, el programa consiste en sumar 21 puntos o quedarse lo más cerca posible, ganando a la banca. Si el jugador se pasa (>21 puntos), pierde y siempre gana la banca.

```
*** BIENVENIDO AL FACKEJACK ***  
Tu carta es: 7  
Puntuación actual: 7  
-----  
¿Quieres otra? (S/N): s
```

En caso de elegirse recibir otra carta, el programa la mostrará de nuevo y actualizará el valor de la puntuación acumulada:

```
*** BIENVENIDO AL FACKEJACK ***
Tu carta es: 7
Puntuación actual: 7
-----
¿Quieres otra? (S/N): s

Tu carta es: 4
Puntuación actual: 11
-----
¿Quieres otra? (S/N): s
```

El programa seguirá ejecutándose de la misma manera mientras el usuario siga pidiendo cartas.

Salida del programa

Si en algún momento el jugador supera los 21 puntos, el programa finalizará automáticamente mostrando por pantalla ***“TE HAS PASADO. LA BANCA GANA!”***

```
*** BIENVENIDO AL FACKEJACK ***
Tu carta es: 7
Puntuación actual: 7
-----
¿Quieres otra? (S/N): s

Tu carta es: 4
Puntuación actual: 11
-----
¿Quieres otra? (S/N): s

Tu carta es: 11
Puntuación actual: 22
-----

TE HAS PASADO. LA BANCA GANA!
```

Si el jugador se planta (responde que no quiere más cartas), el programa debe comparar su puntuación con la generada para la banca al inicio del programa.

- Si la puntuación del jugador es mayor a la de la banca, el programa imprimirá ***“HAS GANADO!”*** junto a las puntuaciones de cada uno:

```

*** BIENVENIDO AL FACHEJACK ***
Tu carta es: 7
Puntuación actual: 7
-----
¿Quieres otra? (S/N): s

Tu carta es: 4
Puntuación actual: 11
-----
¿Quieres otra? (S/N): s

Tu carta es: 8
Puntuación actual: 19
-----
¿Quieres otra? (S/N): n

HAS GANADO!
Puntos banca: 18
Puntos jugador: 19

```

- Si la puntuación del jugador es menor a la de la banca, el programa imprimirá “**HAS PERDIDO! GANA LA BANCA**” junto a las puntuaciones de cada uno:

```

*** BIENVENIDO AL FACHEJACK ***
Tu carta es: 7
Puntuación actual: 7
-----
¿Quieres otra? (S/N): s

Tu carta es: 4
Puntuación actual: 11
-----
¿Quieres otra? (S/N): s

Tu carta es: 6
Puntuación actual: 17
-----
¿Quieres otra? (S/N): n

HAS PERDIDO! GANA LA BANCA
Puntos banca: 18
Puntos jugador: 17

```

Condiciones adicionales a cumplir

Se debe aceptar que el usuario introduzca sus opciones por teclado (más cartas S/N) tanto en mayúsculas como en minúsculas.

(1p) Pega a continuación las capturas de pantalla de tus pruebas.

2. (4,5p) Programa *Suma de vocales*

Cada vez es más común que las aplicaciones de mensajería analicen los mensajes enviados por los usuarios. Le han pedido al equipo de desarrollo de *WhatsApp* crear un programa que cuente cuántas vocales hay en un mensaje, ya que se usará para medir la “expresividad” del texto (los mensajes con más vocales tienden a ser más emocionales).

Dicho programa debe:

1. Pedir al usuario escribir un mensaje de texto (por ejemplo, una frase que enviaría por chat).
2. Contar cuántas vocales no acentuadas que contiene ese mensaje, sin importar si están en mayúscula o minúscula.
3. Mostrar el total de vocales encontradas y cuántas veces aparece cada vocal.

Ejecución de ejemplo:

```
Escribe un mensaje: ¡Hola! ¿Cómo estás hoy?  
-----  
Cantidad total de vocales (no acentuadas): 5  
A: 1  
E: 1  
I: 0  
O: 3  
U: 0
```

- (3,5p) Traduce a lenguaje *Java* la solución al problema planteado.

Pruebas

(1p) Pega a continuación las capturas de pantalla de tus pruebas: